

Materia

Seguridad en el Desarrollo de Aplicaciones

Nombre

Ricardo Porras Reyes

Matricula

2024171008

Grupo

IDSG17

Profesor

Gerardo Ramírez Villarreal

Tabla comparativa de las 10 vulnerabilidades OWASP Top 10.....	3
Casos reales de vulnerabilidades.....	5
Broken Access Control.....	5
Cryptographic Failures.....	5
Injection.....	5
Insecure Design.....	5
Security Misconfiguration.....	5
Vulnerable and Outdated Components.....	6
Identification and Authentication Failures.....	6
Software and Data Integrity Failures.....	6
Security Logging and Monitoring Failures.....	6
Server-Side Request Forgery (SSRF).....	6
Código vulnerable vs código seguro.....	7
SQL Injection — Vulnerable.....	7
SQL Injection — Seguro.....	7
Broken Access Control — Vulnerable.....	7
Broken Access Control — Seguro.....	7
Cryptographic Failures — Vulnerable.....	7
Cryptographic Failures — Seguro.....	8
Vulnerabilidades más críticas.....	8
Injection.....	8
Broken Access Control.....	8
Vulnerable and Outdated Components.....	8
Cryptographic Failures.....	8
Security Misconfiguration.....	8
Conclusión.....	8

Tabla comparativa de las 10 vulnerabilidades OWASP Top 10

Vulnerabilidad	Descripción corta	Ejemplo de ataque	Impacto potencial	Método de mitigación
Broken Access Control	Fallas que permiten acceder a recursos sin autorización.	Un usuario modifica una URL para ver datos de otro usuario.	Robo de información, modificación de datos, escalación de privilegios.	Validar permisos en servidor, usar control de acceso basado en roles y autenticación segura.
Cryptographic Failures	Uso incorrecto o ausencia de cifrado.	Contraseñas almacenadas en texto plano.	Exposición de datos sensibles y robo de credenciales.	Utilizar HTTPS, algoritmos seguros y hash con sal.
Injection	Entrada maliciosa enviada a intérpretes como SQL o comandos.	SQL Injection en formularios de login.	Acceso no autorizado a bases de datos.	Queries parametrizadas y validación de entradas.
Insecure Design	Problemas de diseño de seguridad desde la arquitectura.	Sistema sin límites de intentos de login.	Abuso de funciones críticas y ataques automatizados.	Modelado de amenazas y diseño seguro desde el inicio.
Security Misconfiguration	Configuraciones inseguras en servidores o aplicaciones.	Panel administrativo expuesto públicamente.	Acceso indebido y ejecución de ataques.	Configuración mínima necesaria y actualizaciones constantes.

Vulnerable and Outdated Components	Uso de librerías o software con vulnerabilidades conocidas.	Framework desactualizado o vulnerable a RCE.	Compromiso total del sistema.	Mantener dependencias actualizadas y monitoreadas.
Identification and Authentication Failures	Errores en autenticación o manejo de sesiones.	Contraseñas débiles y sesiones sin expiración.	Secuestro de cuentas.	MFA, políticas de contraseñas y manejo seguro de sesiones.
Software and Data Integrity Failures	Falta de verificación de integridad de software o datos.	Actualizaciones manipuladas con malware.	Ejecución de código malicioso.	Firmas digitales y validación de integridad.
Security Logging and Monitoring Failures	Falta de registros y monitoreo de eventos.	Intrusión no detectada durante semanas.	Respuesta tardía a incidentes.	Logs centralizados y monitoreo continuo.
Server-Side Request Forgery (SSRF)	El servidor realiza solicitudes manipuladas por un atacante.	Acceso interno mediante URL maliciosa.	Acceso a redes internas y robo de datos.	Validar URLs y restringir conexiones salientes.

Casos reales de vulnerabilidades

Broken Access Control

Caso 1 – Facebook (2018):

La vulnerabilidad consistió en la exposición de tokens de acceso. Los atacantes aprovecharon fallas en los controles de acceso para obtener tokens de autenticación de millones de usuarios y acceder a sus cuentas sin autorización.

Caso 2 – Uber (2016):

El ataque ocurrió debido a permisos mal configurados dentro de los sistemas internos de la empresa, lo que permitió el acceso indebido a información sensible.

Cryptographic Failures

Caso 1 – Equifax (2017):

Información sensible fue almacenada y transmitida de manera insegura, permitiendo la exposición masiva de datos personales de millones de usuarios.

Caso 2 – Adobe (2013):

Las contraseñas de usuarios fueron comprometidas debido al uso de métodos de cifrado débiles y malas prácticas de protección de credenciales.

Injection

Caso 1 – Sony Pictures (2011):

Ataques de tipo SQL Injection permitieron a los atacantes acceder y comprometer millones de registros almacenados en las bases de datos de la empresa.

Caso 2 – TalkTalk (2015):

Una vulnerabilidad de SQL Injection permitió el acceso no autorizado a información personal y financiera de clientes.

Insecure Design

Caso 1 – Twitter (2020):

Deficiencias en el diseño de privilegios internos y ataques de ingeniería social permitieron el secuestro de cuentas verificadas de alto perfil.

Caso 2 – Colonial Pipeline (2021):

Un diseño inseguro en los accesos remotos facilitó un ataque de ransomware que afectó operaciones críticas de la empresa.

Security Misconfiguration

Caso 1 – MongoDB (2017):

Miles de bases de datos quedaron expuestas públicamente debido a configuraciones inseguras y ausencia de autenticación.

Caso 2 – Capital One (2019):

Configuraciones incorrectas en servicios en la nube permitieron accesos no autorizados a información confidencial.

Vulnerable and Outdated Components

Caso 1 – Apache Software Foundation (2021):

La vulnerabilidad conocida como Log4Shell en Log4j permitió la ejecución remota de código en múltiples sistemas alrededor del mundo.

Caso 2 – Equifax (2017):

El uso de una versión vulnerable de Apache Struts facilitó uno de los ataques más grandes relacionados con filtración de datos.

Identification and Authentication Failures

Caso 1 – Yahoo (2014):

Debilidades en los mecanismos de autenticación provocaron el robo masivo de cuentas de usuarios.

Caso 2 – Zoom (2020):

La falta de medidas adecuadas de protección en reuniones permitió accesos no autorizados y problemas de seguridad conocidos como “Zoombombing”.

Software and Data Integrity Failures

Caso 1 – SolarWinds (2020):

Atacantes comprometieron las actualizaciones oficiales del software Orion, distribuyendo código malicioso a múltiples organizaciones.

Caso 2 – Codecov (2021):

La modificación maliciosa de scripts utilizados por la plataforma permitió el robo de credenciales y datos sensibles.

Security Logging and Monitoring Failures

Caso 1 – Target (2013):

Aunque existían alertas de seguridad, estas no fueron atendidas correctamente, permitiendo el robo de millones de tarjetas bancarias.

Caso 2 – Marriott International (2018):

Una intrusión persistente permaneció sin ser detectada durante años, comprometiendo información de huéspedes.

Server-Side Request Forgery (SSRF)

Caso 1 – Capital One (2019):

Una vulnerabilidad SSRF permitió a los atacantes acceder a recursos internos alojados en AWS y obtener información confidencial.

Caso 2 – Microsoft (2021):

Microsoft Exchange Server presentó vulnerabilidades relacionadas con SSRF que permitieron comprometer servidores corporativos.

Código vulnerable vs código seguro

SQL Injection — Vulnerable

```
// Código vulnerable

const query = "SELECT * FROM usuarios WHERE usuario = '" +
usuario + "'" AND password = '" + password + "'";

db.execute(query);
```

SQL Injection — Seguro

```
// Código seguro con parámetros

const query = "SELECT * FROM usuarios WHERE usuario = ? AND password =
?";

db.execute(query, [usuario, password]);
```

Broken Access Control — Vulnerable

```
// Código vulnerable

app.get("/admin", (req, res) => {

    res.send("Panel de administrador");

});
```

Broken Access Control — Seguro

```
// Código seguro

app.get("/admin", verificarAdmin, (req, res) => {

    res.send("Panel de administrador");

});
```

Cryptographic Failures — Vulnerable

```
// Contraseña guardada en texto plano

guardarUsuario(usuario, password);
```

Cryptographic Failures — Seguro

```
// Uso de hash seguro

const bcrypt = require('bcrypt');

const hash = bcrypt.hashSync(password, 10);

guardarUsuario(usuario, hash);
```

Vulnerabilidades más críticas

Injection

Considero que es una de las más peligrosas porque permite obtener acceso directo a bases de datos y sistemas críticos. Además, sigue siendo muy común en aplicaciones mal desarrolladas.

Broken Access Control

Una mala validación de permisos puede permitir que cualquier usuario acceda a información privada o funciones administrativas sensibles.

Vulnerable and Outdated Components

Muchas empresas dependen de librerías externas y no actualizarlas puede abrir puertas a ataques conocidos y automatizados.

Cryptographic Failures

La protección incorrecta de datos sensibles puede causar filtraciones masivas de información financiera o personal.

Security Misconfiguration

Errores pequeños de configuración pueden dejar completamente expuestos servidores, bases de datos o servicios internos.

Conclusión

El OWASP Top 10 es una de las principales referencias para entender las vulnerabilidades más comunes dentro del desarrollo de aplicaciones web. Como estudiantes y futuros desarrolladores, muchas veces nos enfocamos en que un sistema funcione correctamente y cumpla con los requisitos solicitados, pero también es muy importante considerar la seguridad desde el inicio del desarrollo. Conocer este tipo de vulnerabilidades ayuda a

identificar errores que pueden comprometer información sensible, afectar la privacidad de los usuarios o incluso provocar pérdidas económicas y daños a una empresa.

Durante esta investigación pude observar que muchas de estas vulnerabilidades siguen presentes incluso en empresas grandes y reconocidas, lo que demuestra que la seguridad informática es un aspecto que debe mantenerse y actualizarse constantemente.

Vulnerabilidades como SQL Injection, fallas de autenticación o malas configuraciones continúan siendo explotadas debido a errores de programación, falta de validaciones o el uso de componentes desactualizados.

En mi formación y futura práctica profesional considero importante aplicar medidas de seguridad como la validación correcta de datos, el uso de autenticación segura, el control de accesos y el cifrado de información sensible. Además, también es necesario mantener actualizadas las dependencias y contar con monitoreo y registros de actividad para detectar posibles incidentes a tiempo. En conclusión, estudiar el OWASP Top 10 no solo ayuda a desarrollar aplicaciones más seguras, sino también a crear una mentalidad más preventiva y responsable respecto a la protección de la información y la confianza de los usuarios.